

Vibe Coding: Building & Deploying an AI Web Application on AWS (Project Work)

Objective

This deliverable aims to assess students' ability to apply AI-assisted development techniques and cloud infrastructure skills gained across the Vibe Coding Masterclass Series. Learners will conceptualize, build, and publicly deploy a full-stack AI-powered web application using prompt engineering, an AI-native coding environment, and Amazon Web Services (AWS), demonstrating their ability to function as architects and creative directors in an AI-first development workflow.

Project Brief

You are required to design and ship a functional, publicly accessible AI web application — built entirely through Vibe Coding. The application must include a working frontend interface, a backend server that connects securely to a Large Language Model (LLM) API, and must be containerized and deployed live on AWS cloud infrastructure.

While certain components are required, learners are encouraged to choose their own application concept and explore creative use cases for AI. The emphasis is on demonstrating a complete development lifecycle — from idea to live product — using the tools, frameworks, and prompting strategies learned throughout the course.

Suggested Application Themes

You may build any of the following types of applications or develop your own concept:

- AI Chat Interface: A conversational assistant built around a specific domain (customer support, education, wellness, finance, etc.)
- Content Generation Tool: An application that generates text, summaries, emails, or creative writing based on user input.
- Document Analyzer: Upload a PDF or paste text and receive AI-powered analysis, extraction, or rewriting.
- AI Recommendation Engine: A tool that takes user preferences and returns personalized suggestions (movies, books, career paths, etc.)
- Productivity Assistant: Task manager, meeting summarizer, or daily planner powered by an LLM.
- Learning & Quiz Tool: An AI tutor or quiz generator built around a subject of choice.
- Data Explainer: Input raw data or a CSV and receive plain-language insights generated by an AI model.
- Multilingual Tool: Translation, localization, or language-learning assistant.
- Custom GPT-Style Agent: A role-specific AI persona (legal assistant, fitness coach, recipe generator, etc.)

Application Requirements

Your application must include:

- A functional frontend (HTML/CSS/JavaScript or equivalent framework) with a clear, responsive user interface optimized for both desktop and mobile.
- A working backend server (Python FastAPI or Node.js Express) that handles API routing and keeps all secret keys secure on the server side.
- LLM API integration: The backend must connect to an external AI model (Anthropic Claude, OpenAI GPT, or equivalent) and return live responses to the user.

- Streaming or real-time response: Text should render progressively in the UI rather than appearing all at once after a delay.
- Containerization: The full-stack application must be packaged into a Docker container with a working Dockerfile.
- Live AWS deployment: The containerized application must be deployed to AWS (App Runner, Elastic Beanstalk, or equivalent) and accessible via a public HTTPS URL.
- Security best practices: API keys must not appear in frontend code or version control. Environment variables must be used for all sensitive credentials.

Deliverables

1. Project Concept Note (PDF)

The concept note should clearly describe:

- Project title and application name
- Problem statement / objective
- Target user and use case
- LLM model and API used
- Key features of the application
- Expected user experience and outcomes

2. Live Application

Ensure the application is publicly accessible via its AWS URL and is fully functional at the time of submission.

2.1

Paste your live AWS application URL within the Project Report and Concept Note.

3. Project Report

Submit a written report documenting the full development process. The report must cover:

- Application overview and tech stack
- Prompting strategy and frameworks used (with sample prompts)
- Phase-by-phase development summary
- Application architecture
- Challenges encountered and how they were resolved
- Key learnings and reflection

Evaluation Criteria

Criteria	Weightage
Technical Implementation & Vibe Coding Methodology	25%
Prompt Engineering Quality & Documentation	20%
Cloud Deployment & AWS Architecture	20%
Application Design & User Experience	20%
Report Quality, Reflection & Clarity	15%

Additional Guidelines

- **Application Concept:** You may build any application relevant to your personal, professional, or academic interests. The concept must be original and developed during this course.
- AI tools are permitted and encouraged for all code generation, debugging, and design, this is the entire point of Vibe Coding. Document the tools and prompts you used.
- Ensure your application is functional and accessible on both desktop and mobile browsers before submission.
- You may collaborate in teams of 2–3, but each team member must be able to explain all technical and design decisions independently if asked.
- Maintain academic honesty. Prompt engineering, architectural decisions, and documentation must reflect your own thinking. Submitting another student's application or a pre-built template without meaningful original development is not permitted.
- **Cost awareness:** Ensure AWS budget alerts are configured to avoid unexpected charges. Free tier services are sufficient for this project.